

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
Требования к реализации.....	5
Алгоритм работы программы.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм метода setHP класса Character.....	9
3.2 Алгоритм метода getHP класса Character.....	9
3.3 Алгоритм метода changeHP класса Character.....	10
3.4 Алгоритм функции processByPointer.....	10
3.5 Алгоритм функции processByPointer.....	10
3.6 Алгоритм функции main.....	11
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	15
5.1 Файл Character.cpp.....	15
5.2 Файл Character.h.....	15
5.3 Файл main.cpp.....	16
6 ТЕСТИРОВАНИЕ.....	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	18

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программную систему, моделирующую игрового персонажа и демонстрирующую различие между передачей объекта в функцию по указателю и по значению.

Программная система содержит **один класс**. Класс должен включать:

- одно закрытое целочисленное свойство значения здоровья персонажа;
- открытые методы установки и получения значения здоровья;
- открытый метод изменения значения здоровья (значение параметра прибавляется к значению здоровья).

Требования к реализации

1. В программе должна быть реализована функция, принимающая **указатель на объект**.
2. В программе должна быть реализована функция, принимающая **объект**.
3. В обеих функциях должно использоваться одинаковое условие изменения значения здоровья:
 - о если значение здоровья меньше 50, здоровье увеличивается на 10;
 - о если значение здоровья больше или равно 50, здоровье уменьшается на 5.
4. Освобождение динамически выделенной памяти является обязательным.
5. Использование передачи по ссылке запрещено.

Алгоритм работы программы

1. Динамически создать объект класса.

2. Ввести начальное значение здоровья.
3. Установить значение здоровья через метод класса.
4. Передать указатель на объект в функцию.
5. Передать объект в функцию.
6. Вывести итоговое значение здоровья.
7. Освободить выделенную память.
8. Завершить работу программы.

1.1 Описание входных данных

Одно целое число — начальное значение здоровья персонажа.

1.2 Описание выходных данных

HealthPoint: "Одно целое число — итоговое значение здоровья персонажа".

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект hero класса Character предназначен для хранения здоровья игрового персонажа и демонстрации различий между передачей объекта в функцию по указателю и значению;
- функция main для организации работы программы: динамического создания объекта, ввода начального здоровья, вызова функций с разными способами передачи, вывода результата и освобождения памяти;
- функция processByPointer для демонстрации передачи объекта по указателю (изменения сохраняются в оригинале);
- функция processByValue для демонстрации передачи объекта по значению (изменения не влияют на оригинал);
- оператор new - динамическое выделение памяти;
- оператор delete - освобождение динамически выделенной памяти;
- указатель для хранения адреса динамически созданного объекта;
- оператор * для разыменования указателя.

Класс Character:

- свойства/поля:
 - поле хранение текущего значения здоровья персонажа:
 - наименование — healthPoints;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - метод setHP — устанавливает значение закрытого свойства healthPoints;
 - метод getHP — возвращает текущее значение здоровья;

- о метод `changeHP` — изменяет значение здоровья на указанную величину.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода setHP класса Character

Функционал: устанавливает значение закрытого свойства healthPoints.

Параметры: цел hp - входное значение для установки здоровья персонажа.

Возвращаемое значение: метод ничего не возвращает.

Алгоритм метода представлен в таблице 1.

Таблица 1 – Алгоритм метода setHP класса Character

№	Предикат	Действия	№ перехода
1		присвоить полю healthPoints значение параметра hp	Ø

3.2 Алгоритм метода getHP класса Character

Функционал: возвращает текущее значение здоровья.

Параметры: нет.

Возвращаемое значение: цел - текущее значение здоровья персонажа.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода getHP класса Character

№	Предикат	Действия	№ перехода
1		вернуть значение healthPoints	Ø

3.3 Алгоритм метода changeHP класса Character

Функционал: изменяет значение здоровья на указанную величину.

Параметры: цел delta - значение, на которое изменяется здоровье.

Возвращаемое значение: метод ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода changeHP класса Character

№	Предикат	Действия	№ перехода
1		прибавить delta к значению healthPoints	Ø

3.4 Алгоритм функции processByPointer

Функционал: демонстрация передачи объекта по указателю (изменения сохраняются в оригинале).

Параметры: Character* с - указатель на объект Character.

Возвращаемое значение: функция не возвращает значения.

Алгоритм функции представлен в таблице 4.

Таблица 4 – Алгоритм функции processByPointer

№	Предикат	Действия	№ перехода
1	с->getHP() < 50	вызвать метод с->changeHP с параметром 10	Ø
		вызвать метод с->changeHP с параметром -5	Ø

3.5 Алгоритм функции processByPointer

Функционал: демонстрация передачи объекта по указателю (изменения не влияют на оригинал).

Параметры: Character с - объект Character, передаваемый по значению.

Возвращаемое значение: функция не возвращает значения.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции *processByPointer*

№	Предикат	Действия	№ перехода
1	c.getHP() < 50	вызвать метод c.changeHP с параметром 10	Ø
		вызвать метод c.changeHP с параметром -5	Ø

3.6 Алгоритм функции *main*

Функционал: организация работы программы: динамическое создание объекта, ввод начального здоровья, вызов функций с разными способами передачи, вывод результата и освобождение памяти.

Параметры: нет.

Возвращаемое значение: цел - код возврата программы.

Алгоритм функции представлен в таблице 6.

Таблица 6 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявл. цел. переменной initHP	2
2		ввод значения initHP с клавиатуры	3
3		динамическое создание объекта Character с помощью new, сохранение адреса в указателе hero	4
4		вызов метода setHP с параметром initHP для объекта hero	5
5		вызов функции processByPointer с указателем на объект hero в качестве параметра	6
6		вызов функции processByValue с передачей разыменованного указателя *hero	7
7		вывод на экран "HealthPoints: ", hero->getHP(), endl	8
8		освободить динамическую память для hero	9
9		возврат 0	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-3.

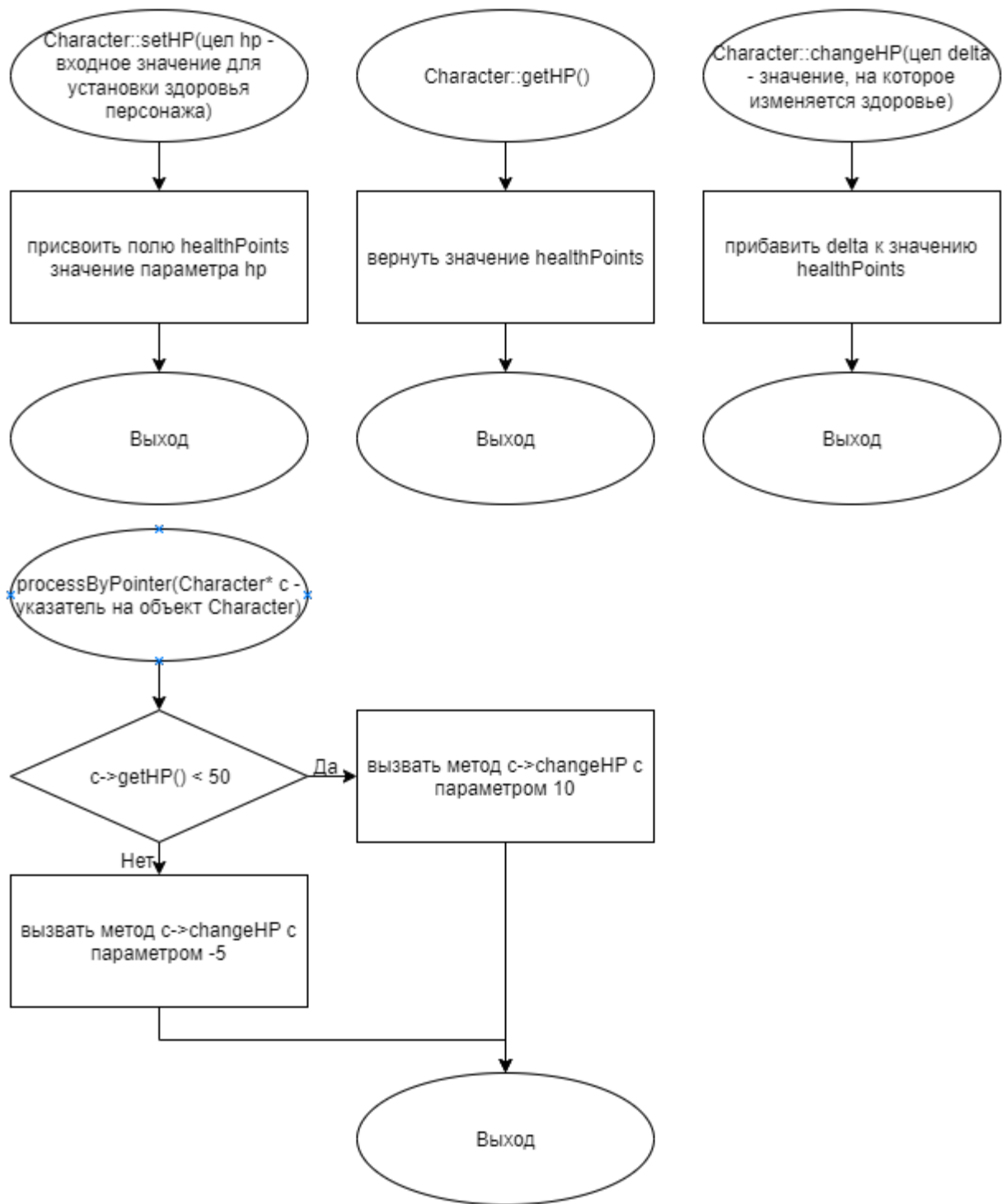


Рисунок 1 – Блок-схема алгоритма

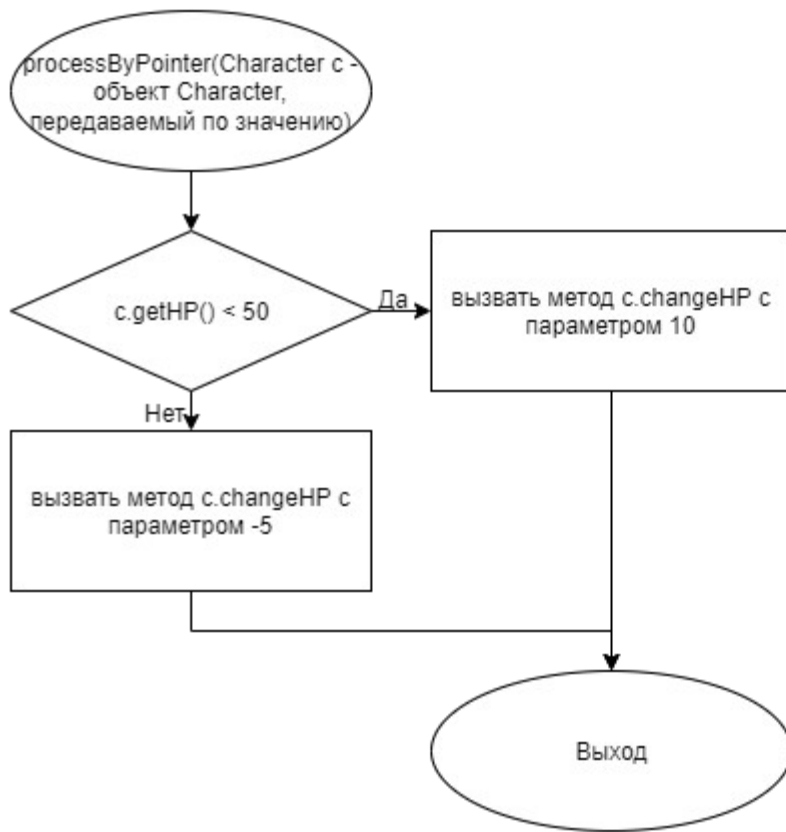


Рисунок 2 – Блок-схема алгоритма

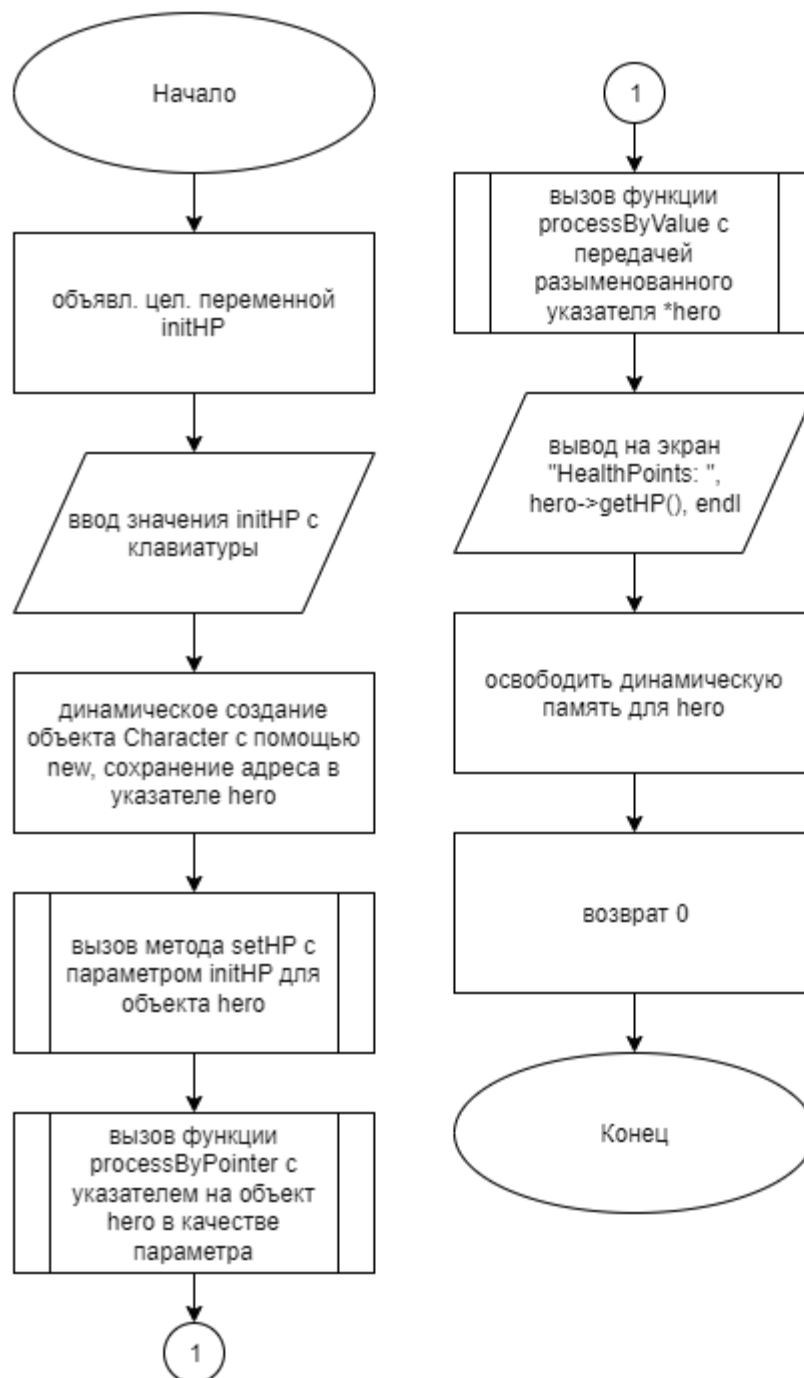


Рисунок 3 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Character.cpp

Листинг 1 – Character.cpp

```
#include "Character.h"

void Character::setHP(int hp) {
    healthPoints = hp;
}

int Character::getHP() const {
    return healthPoints;
}

void Character::changeHP(int delta) {
    healthPoints += delta;
}
```

5.2 Файл Character.h

Листинг 2 – Character.h

```
#ifndef __CHARACTER__H
#define __CHARACTER__H

class Character {
    int healthPoints;
public:
    void setHP(int hp);
    int getHP() const;
    void changeHP(int hp);
};

#endif
```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include "Character.h"

void processByPointer(Character* c) {
    if (c->getHP() < 50) {
        c->changeHP(10);
    } else {
        c->changeHP(-5);
    }
}

void processByValue(Character c) {
    if (c.getHP() < 50) {
        c.changeHP(10);
    } else {
        c.changeHP(-5);
    }
}

int main()
{
    int initHP;
    std::cin >> initHP;
    Character* hero = new Character();
    hero->setHP(initHP);
    processByPointer(hero);
    processByValue(*hero);
    std::cout << "HealthPoints: " << hero->getHP() << std::endl;
    delete hero;
    return 0;
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 7.

Таблица 7 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
49	HealthPoints: 59	HealthPoints: 59
50	HealthPoints: 45	HealthPoints: 45

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).